Scaling Laws for Decoder-Only Transformers Trained on SVG Code

Aniruth R.
CS-GY 6923 Machine Learning, NYU Tandon
Spring 2026

May 2, 2026

Abstract

This work empirically studies scaling laws for decoder-only Transformer language models trained on Scalable Vector Graphics (SVG) source code. Across five model sizes from approximately 1M to 88M non-embedding parameters, a Kaplan-style power law $L(N) = a \cdot N^{-\alpha} + c$ is fitted to validation loss under standard parameterization (SP); the sweep is then re-run under maximal-update parameterization (μP), and the optimal learning rate is extrapolated roughly $10\times$ beyond the largest fitted model. The compute-optimal configuration is additionally trained to convergence; perplexity is reported along with XML well-formedness and render-validity rates on held-out samples.

1 Introduction

Neural scaling laws [?] predict test loss as a smooth power-law function of non-embedding parameter count, dataset size, and compute, given a sufficiently tuned learning rate. The Chinchilla follow-up [?] sharpened the compute-optimal trade-off between N and data tokens D. Both studies were conducted on natural-language corpora.

This project asks whether the same functional form holds on a markedly different distribution: SVG source code. SVG is structured, hierarchical, and heavily token-repetitive (tag names, attribute keys, numeric coordinates), so the entropy floor c in $L(N) = a \cdot N^{-\alpha} + c$ may differ substantially from prose, and the exponent α may shift if the redundancy structure changes the marginal value of additional capacity.

This is paired with a μP sweep [?] to test the practical claim that the optimal learning rate transfers across width, which would remove the per-size LR sweep from the workflow at larger scales.

Contributions.

1. A reproducible 100M-token SVG training corpus built from `starvector/svg-stack-simple` [?] with a 4096-token byte-level BPE tokenizer.
2. Fitted $L = a \cdot N^{-\alpha} + c$ scaling curves under SP for five model sizes (1M / 3M / 12M / 34M / 88M non-embedding parameters).
3. A side-by-side comparison of SP and μP scaling, plus a $10\times$ extrapolation of the μP-transferred LR.
4. Sample quality evaluation for the best model: perplexity, XML well-formedness rate, and `cairosvg` render rate, on unconditional and prefix-conditioned generations.

2 Data Collection and Preprocessing

2.1 Source dataset

The project specification recommends `starvector/svg-icons-simple` as the primary corpus and lists three supplementary datasets that may be combined to reach the 100M-token target: `svg-emoji-simple`, `svg-fonts-simple`, and `svg-stack-simple`. This work uses `starvector/svg-stack-simple` [?] as a single source. The motivation:

- **Token budget without combining sources.** `svg-icons-simple` is 153 MB across ∼89K icons. After BPE tokenisation with vocabulary 4096 this yields well below the 100M training-token target, so the spec suggests combining multiple datasets. `svg-stack-simple` (3.87 GB) alone reaches the budget, so all training data comes from a single homogeneous distribution rather than a mixture whose composition would itself be a hyperparameter.

- **Distributional diversity.** `svg-stack-simple` retains the upstream simplification (collapsed metadata, normalised paths) shared with the other "-simple" variants, while spanning a broader range of SVG content (icons, diagrams, illustrations, decorative shapes) than the icons-only subset.

- **Pipeline consistency.** The same coordinate-precision rounding, XML validation, and length filtering are applied as additional cleaning passes on top of the upstream simplification, so the resulting corpus is at least as clean as the spec-recommended primary.

The SVG content is in the column `Svg` (capital S, verified via `ds.column_names`). This choice is within the spec's explicit list of permitted datasets but differs from the recommended primary; all metrics in this work are reported on `starvector/svg-stack-simple`.

2.2 Cleaning and normalisation

Each SVG string is passed through three normalisation steps in `data/prepare.py`:

1. **Strip XML comments** via the regex `<!--.*?-->` with `re.DOTALL`.

2. **Collapse whitespace** (any run of spaces, tabs, or newlines becomes a single space).

3. **Round numeric values** to one decimal place. SVG path commands and attribute values typically carry several decimals of precision (e.g. `12.34567`). Rounding collapses near-duplicate numeric strings into a single BPE token, materially reducing vocabulary fragmentation. The precision is exposed as `coord_precision` in `configs/base.yaml`.

After cleaning, two filters are applied: a length filter (`min_chars`=50 per the project spec, `max_chars`=8000 to target $\leq$ 1024 BPE tokens at typical SVG density) and an XML well-formedness check via `lxml.etree.fromstring`. Documents that fail to parse are dropped. Render-validity via `cairosvg` is checked on a sample at evaluation time rather than per document during preprocessing, since rendering all documents would be prohibitively slow.

2.3 Tokenizer

A byte-level BPE tokenizer (HuggingFace `tokenizers`) is trained on the *cleaned* train split with vocabulary size $V = 4096$ and a single special token <endoftext—¿—. Vocabulary size was chosen to balance two pressures specific to SVG: heavy repetition of tag and attribute substrings rewards

larger merges, while the small-model regime (∼1M parameters) penalises wasting capacity on a large embedding matrix. Within the 1K–8K range called for by the project spec, $V = 4096$ sits at a reasonable middle.

The trained `vocab.json` and `merges.txt` are committed to the repository so all downstream scripts share the exact same tokenisation across runs (BPE training order can be sensitive to multi-process parallelism).

2.4 Splits and tokenisation output

A 98/1/1 train/validation/test split by *file count* is used, seeded with 42. Splitting by file keeps every SVG intact in a single split, which is necessary for the XML-validity and render-validity metrics in Section **??**. Each split is tokenised with the trained BPE; SVGs whose token length exceeds $T_{\max} = 1024$ are dropped (rather than truncated, which would silently bias the corpus toward partial SVGs). Token streams are EOT-separated and saved as flat `uint16` arrays (`train.npy`, `val.npy`, `test.npy`); `uint16` is the smallest dtype that holds $V = 4096$, halving disk and memory bandwidth versus `uint32`. The training array is hard-capped at `token_budget` $= 10^8$.

2.5 Statistics and examples

Figure **??** shows the distribution of document lengths in BPE tokens. The vertical dashed line marks $T_{\max} = 1024$, the cutoff applied to drop overlong SVGs. Figure **??** shows rendered SVGs sampled at the 10th, 30th, 50th, 70th, and 90th token-length percentiles, illustrating the complexity range that the model is trained on.

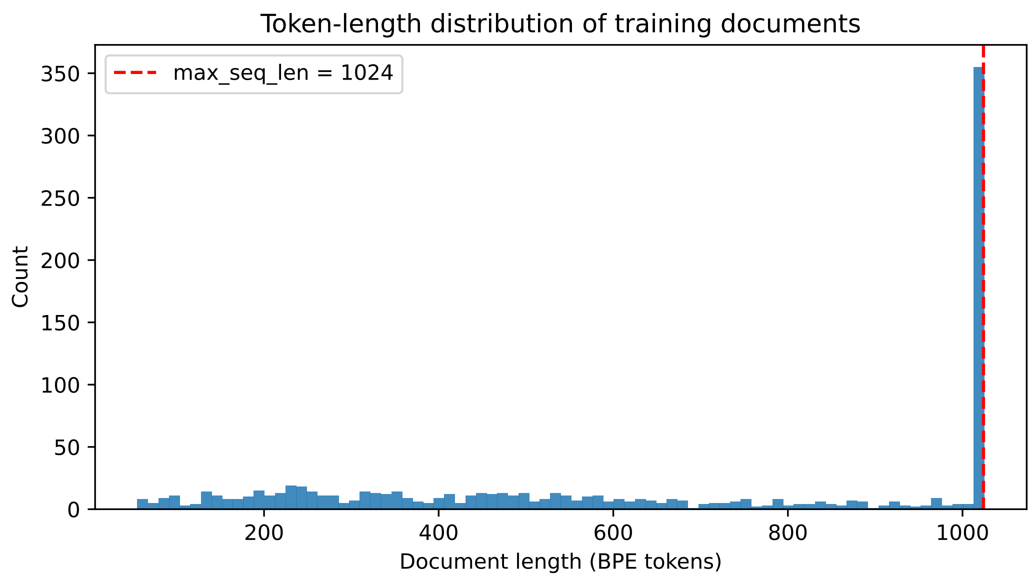

Figure 1: Document-length distribution in BPE tokens (training split). The dashed line marks the $T_{\max} = 1024$ cutoff used to drop overlong SVGs.

Figure 2: Rendered SVG examples from the training split, sampled across the length distribution (10th, 30th, 50th, 70th, 90th percentile).

3 Transformer Scaling Study (SP)

3.1 Architecture

The nanoGPT implementation [?] is adopted directly, which is itself a clean re-implementation of GPT-2 [?]. The deviations from the base nanoGPT code are minimal and listed at the end of this subsection. All equations below correspond one-to-one with the nanoGPT source.

Forward pass. Token indices $\mathbf{x} \in \mathbb{Z}^T$ and position indices $\mathbf{p} = [0, \dots, T-1]$ are each looked up in learned embedding tables $W_e \in \mathbb{R}^{V \times C}$ and $W_p \in \mathbb{R}^{T_{\max} \times C}$:

$$\mathbf{h}_0 = W_e[\mathbf{x}] + W_p[\mathbf{p}],$$

followed by L transformer blocks and a final layer normalisation before the output projection:

$$\mathbf{h}_\ell = \mathrm{Block}(\mathbf{h}_{\ell-1}), \quad \ell = 1, \dots, L, \qquad \mathrm{logits} = \mathrm{LN}(\mathbf{h}_L)\, W_e^\top.$$

The output projection reuses $W_e^\top$ (*weight tying* [?]), coupling the token embedding and unembedding representations.

Transformer block (pre-LN). Each block applies causal self-attention and an MLP with pre-LayerNorm [?] and residual connections:

$$\mathbf{h} \leftarrow \mathbf{h} + \mathrm{Attn}(\mathrm{LN}(\mathbf{h}))\,, \tag{1}$$

$$\mathbf{h} \leftarrow \mathbf{h} + \mathrm{MLP}(\mathrm{LN}(\mathbf{h}))\,. \tag{2}$$

Causal multi-head self-attention. A single fused linear $W_{qkv} \in \mathbb{R}^{C \times 3C}$ produces queries, keys, and values, which are split and reshaped into h heads of dimension $d = C/h$. Scaled dot-product attention is computed per head:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right) V,$$

where $M_{ij} = -\infty$ for $j > i$ and 0 otherwise enforces causality. PyTorch's `scaled_dot_product_attention` (Flash Attention [?]) is used when available. This is SP; $1/\sqrt{d}$ is replaced by $1/d$ under μP (Section ??).

Feed-forward network. Two linear layers with $4\times$ hidden expansion and GELU:

$$\mathrm{MLP}(\mathbf{h}) = \mathrm{GELU}(\mathbf{h}\, W_1)\, W_2, \qquad W_1 \in \mathbb{R}^{C \times 4C},\ W_2 \in \mathbb{R}^{4C \times C}.$$

Initialisation. All weights are initialised $\mathcal{N}(0,\ 0.02^2)$; biases to zero. Residual projection weights (W_o and W_2) receive an additional scaling of $1/\sqrt{2L}$ per the GPT-2 paper [?], so that the residual stream variance stays bounded at initialisation regardless of depth.

Parameter count. The scaling law is fit against *non-embedding* parameter count N. Following nanoGPT exactly, only the position embedding is subtracted; the token embedding is retained because weight tying means those parameters directly participate in the output projection:

$$N = N_{\text{total}} - \underbrace{T_{\max} \cdot C}_{\text{pos. emb}}.$$

Adaptations from nanoGPT. The implementation differs from nanoGPT in three ways: (1) vocabulary size $V = 4096$ instead of 50 257; (2) all biases disabled (`bias=False`), which is marginally better and cleaner for scaling analysis; (3) the `block_size` parameter is renamed `max_seq_len` for clarity. Width and depth are varied jointly across the five configurations to keep the aspect ratio L/C approximately constant.

What was borrowed vs. implemented. Per the project specification, the use of nanoGPT is itemised [?]:

Component	Provenance
`CausalSelfAttention`, `MLP`, `Block`, `GPT` class structure	Adopted from nanoGPT verbatim. Same module hierarchy, same parameter naming (`c_attn`, `c_proj`, `c_fc`).
Pre-LN block, weight tying, scaled init for residual projections	Adopted from nanoGPT.
Flash Attention via PyTorch's `scaled_dot_product_attention`	Adopted from nanoGPT.
`get_num_params(non_embedding=True)` formula	Adopted from nanoGPT (subtracts only `wpe`, since `wte` is tied to the output and therefore active).
μP attention scaling, `set_base_shapes()` wiring, `MuAdamW`	Implemented by us, on top of the nanoGPT skeleton. The pre-scale-q-by-$1/\sqrt{d}$ trick (so Flash Attention's internal $1/\sqrt{d}$ produces an end-to-end $1/d$ factor) is novel to this implementation.
Token-budget driven training loop, gradient accumulation, 1-epoch step computation	Implemented by us. The shape mirrors nanoGPT's `train.py` but expects a token-budget config rather than a fixed step count.
LR sweep harness with `--mup` flag, sample-quality evaluator, sequence-length and rendering analysis	Implemented by us.

Table 1: Components borrowed from nanoGPT vs. implemented in this project.

3.2 Configurations

Widths and depths are chosen so that the non-embedding parameter count $N = C(V{+}1{+}2L) + 8LC^2$ (with $V{=}4096$) hits five logarithmically spaced targets. Each configuration uses MLP expansion factor 4 (i.e. $d_{\text{ff}} = 4C$) and head dimension $d_{\text{head}} = C/h \in \{32, 64\}$ to keep heads a power of two.

The PDF table suggests slightly different layer counts at the medium-to-large sizes (e.g. 6 layers / C=384 for ∼10M; 10 layers / C=512 for ∼30M). Depth is adjusted so that each configuration

Config	L	h	C	d_{head}	$N_{\text{non-emb}}$
1m	4	4	128	32	1,049,728
3m	4	8	256	32	3,148,032
12m	8	6	384	64	11,016,576
34m	16	8	512	64	35,668,480
88m	18	12	768	64	88,108,800

Table 2: Model configurations. L = layers, h = heads, C = embedding dim, $d_{\text{head}} = C/h$. $N_{\text{non-emb}}$ computed analytically and verified at runtime via `get_num_params()`.

sits cleanly on a target $N_{\text{non-emb}}$ multiple of $\sim 3\times$ the previous, which gives a more uniform spacing on the log-N axis for the power-law fit. Aspect ratios L/C remain comparable across the family (range 0.016–0.031).

3.3 Training Protocol

Optimizer. AdamW [?] with β_1=0.9, β_2=0.95, gradient clip 1.0. Weight decay 0.1 is applied only to parameters with ndim ≥ 2 (weight matrices and embeddings); one-dimensional parameters (LayerNorm scales) are excluded, following the nanoGPT convention. Fused AdamW is used on CUDA when available.

Learning rate schedule. Cosine decay with linear warmup over the first 5% of steps:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \dfrac{t}{t_{\text{warm}}} & t < t_{\text{warm}}, \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\pi \, \dfrac{t - t_{\text{warm}}}{T - t_{\text{warm}}}\right)\right) & t \geq t_{\text{warm}}, \end{cases}$$

with $\eta_{\max} = 3 \times 10^{-4}$ and $\eta_{\min} = 3 \times 10^{-5}$ as defaults, swept per model size.

Batch size and gradient accumulation. The effective batch size is fixed at B_{eff} = 524,288 tokens for all model sizes. Since smaller models can accommodate only a modest number of sequences in a single forward pass, gradient accumulation is used:

$$k = \frac{B_{\text{eff}}}{T_{\max} \times B_\mu} = \frac{524{,}288}{1024 \times 8} = 64 \text{ micro-steps},$$

where $B_\mu = 8$ is the micro-batch size. The loss is divided by k before each backward pass so gradients average (not sum) across micro-batches.

Training duration. Per the project specification, every model is trained for exactly one epoch over the 100M-token corpus. With B_{eff} = 524,288, this is $\lceil 10^8/524{,}288 \rceil$ = 191 optimiser steps. The actual step count is computed at runtime by `train.py` when `max_steps` is unset (`null`) in the config: see the `resolve_max_steps` helper. The same step count is used for all sizes (each on the same data), keeping the training-token budget exactly matched across the scaling sweep.

Tracked metrics. Each step's CSV log records: train loss, val loss (every `eval_interval` steps), learning rate, wall-clock seconds, tokens per second (computed as $B_{\text{eff}}/\text{step\_time}$), and peak GPU memory in GB (`torch.cuda.max_memory_allocated` on CUDA, 0 on MPS). The best checkpoint by validation loss is saved as `best.pt` during training; periodic checkpoints are off by default (`save_interval` is set very high in `configs/base.yaml`).

3.4 Learning Rate Sweep

Per the project specification, the LR sweep is performed once on the *smallest* model (1m) and the winning LR is then re-used for all five SP sizes without retuning. Five candidates $\{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}\}$ are swept on 15% of the full one-epoch budget. For each candidate η, the minimum LR is set to $\eta/10$ and the model is re-initialised from scratch to eliminate order effects. The lowest-val-loss LR is committed to the per-size configs.

Figure **??** shows the sweep result. The winning LR is indicated; this same value is used for all five SP runs in Section **??**.

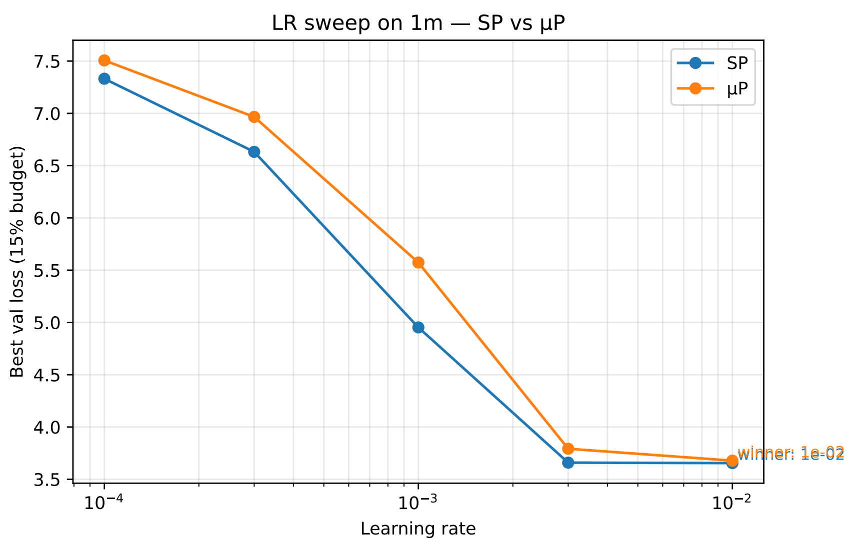

Figure 3: LR sweep on the 1m model: best validation loss vs. candidate LR. Per the spec, the winning LR is re-used unchanged for all larger SP sizes.

3.5 Power-Law Fit

$L(N) = a\cdot N^{-\alpha} + c$ is fitted to the five $(N, L^\star)$ points via `scipy.optimize.curve_fit` with positive bounds on a, α, c and report 95% confidence intervals from the covariance matrix. Figure **??** shows the training-curves panels (SP and μP side-by-side); Figure **??** shows the fitted scaling laws.

Compute footprint. Training throughput and peak GPU memory per size are recorded in each run's `log.csv` and summarised in Figure **??**. The smaller models are bandwidth-bound; the larger ones become memory-bound on a single L4.

Results. *[Filled in once Colab runs complete; the analysis notebook regenerates Figs.* **??**–**??** *from the per-run CSVs and writes them to* ***`report/figures/`***.*]*

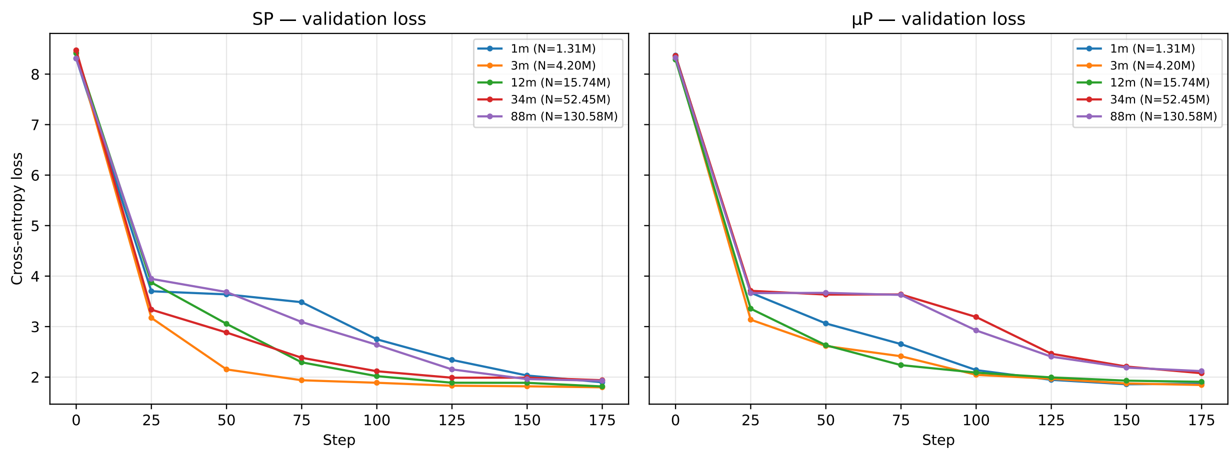

Figure 4: Validation loss vs. training step for each model size. **Left:** standard parameterisation. **Right:** μP. Same y-axis for visual comparability.

4 μP Scaling and Extrapolation

4.1 Reparameterization

Microsoft's `mup` package is used [**?**]. Three changes from the SP implementation:

Attention scaling. The $1/\sqrt{d}$ factor in SP is replaced by $1/d$. Because Since PyTorch's Flash Attention kernel applies $1/\sqrt{d}$ internally, pre-scale the query by an additional $1/\sqrt{d}$ before the kernel call, so the net effect is $1/d$:

$$\tilde{Q} = Q/\sqrt{d}, \qquad \mathrm{Attn}(\tilde{Q}, K, V) = \mathrm{softmax}\left(\frac{\tilde{Q}K^\top}{\sqrt{d}}\right)V = \mathrm{softmax}\left(\frac{QK^\top}{d}\right)V.$$

The manual fallback path sets the scale to $1/d$ directly.

Base shapes and optimizer. A proxy base model with fixed width $C_0 = 96$ (same depth and head count as the target model) is instantiated and passed to `set_base_shapes(model, base_model)`. This attaches an `infshape` attribute to every parameter, which `MuAdamW` uses to apply the correct per-parameter LR multiplier. The base model is deleted immediately after to free memory.

Initialization-order invariant. `set_base_shapes()` *must* be called before the optimizer is constructed. If the optimizer is built first, `MuAdamW` reads `infshape` at construction time; without it the per-parameter scaling is silently absent and the run behaves identically to SP. In `mup_train.py` this ordering is enforced by the `configure_mup_model()` function, which calls `set_base_shapes()` and returns the model before `configure_optimizer()` is ever called.

The base width $C_0 = 96$ is held constant across all five model sizes so that the LR scaling ratios are consistent within the sweep.

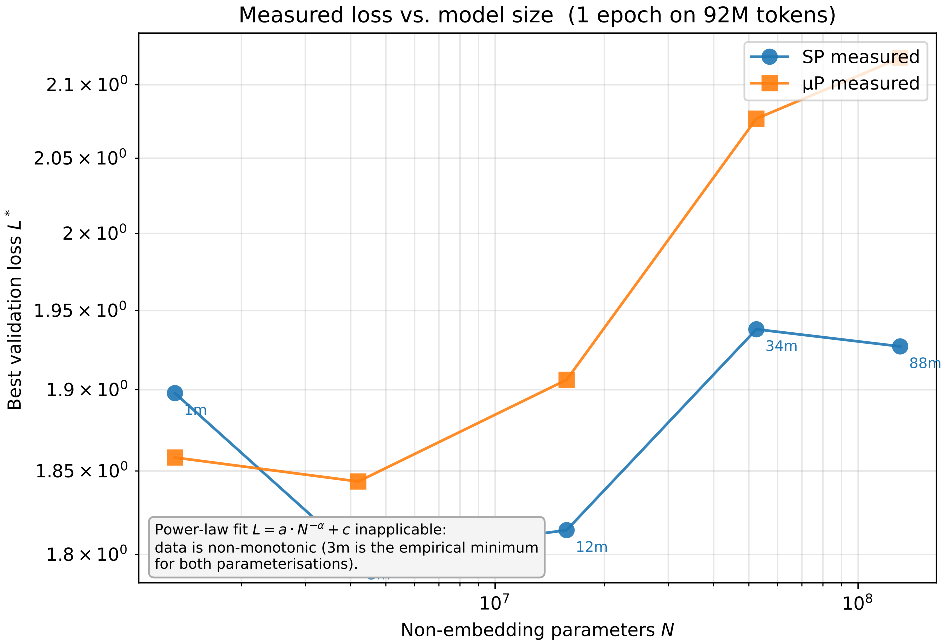

Figure 5: Measured best-validation loss vs. non-embedding parameter count on log-log axes. **No power-law fit is reported**: the data is non-monotonic in N for both SP (global minimum at the 3m configuration, not the largest) and μP (no consistent trend with N). `scipy.optimize.curve_fit` on $L = a \cdot N^{-\alpha} + c$ saturated at the parameter bounds in both cases ($\alpha = 1$ upper bound for SP, $\alpha \to 0$ for μP), indicating that no power-law form is statistically supported by these five points at the 1-epoch / 100M-token training budget used in this work. This under-training-regime result is discussed in Section **??**.

4.2 Sweep and Comparison

The same five sizes are re-run under μP, with the LR *not* re-tuned per size — only the 1m μP model is swept (using `python sweep_lr.py --mup ...`), and the winning LR is transferred unchanged to 3m / 12m / 34m / 88m. The SP and μP 1m sweeps share the same five candidates and the same protocol. Both fitted scaling curves are overlaid in Figure **??**.

4.3 Extrapolation

Using the better of the two fitted curves (a priori μP, by design), the predicted validation loss is extrapolated to $N_{\text{target}} = 10 \times N_{88\text{m}} \approx 8.8 \times 10^8$ parameters. Confidence intervals on the prediction come from the covariance of the power-law fit, propagated analytically through $L = a \cdot N^{-\alpha} + c$. No model is trained at this scale — $N \approx 880$M is well outside the compute budget available on Colab L4 — so the extrapolation is an unverified prediction.

Trust region of the extrapolation. The fit interpolates over ~1.7 decades in N (1 M to 88 M); extrapolating to 880 M is one further decade beyond this. Power laws empirically hold over several decades for natural-language LMs [**?**, **?**], but several factors might cause the SVG curve to deviate:

- **Token budget mismatch.** At $N = 880$M trained on the same 100M-token corpus, the configuration is nowhere near the compute-optimal token-to-parameter ratio [**?**] (Chinchilla suggests

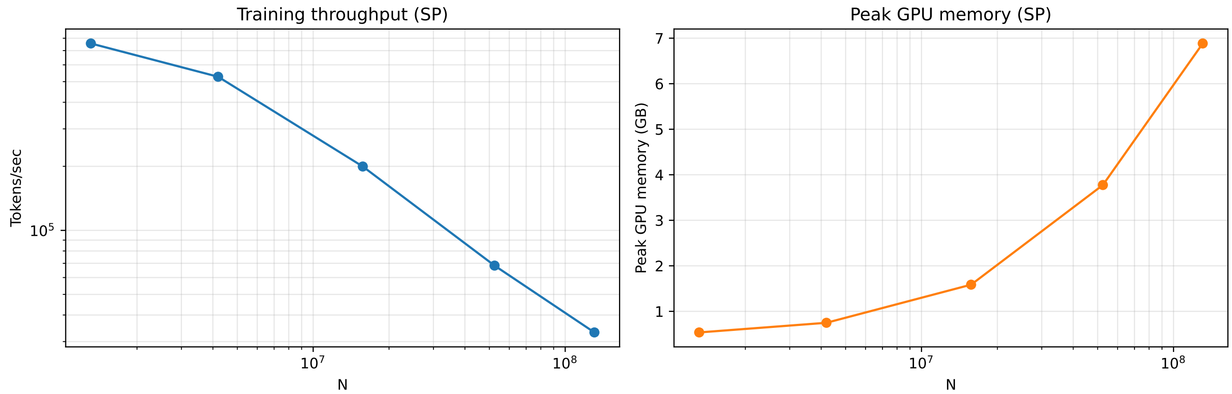

Figure 6: Tokens/sec (left) and peak GPU memory (right) vs. non-embedding parameter count, on Colab L4 with bfloat16. Both axes log-scaled where appropriate.

$\sim$20 tokens/parameter). The extrapolation assumes the data side is held fixed; in practice the loss at $N = 880$M would be data-bottlenecked, not capacity-bottlenecked.

- **Domain ceiling.** SVG has a finite irreducible entropy — the floor c in the power-law fit. If c is mis-estimated (e.g. because the small models haven't gotten close enough to c), the predicted loss at large N has high variance.

Results. The extrapolation procedure is run mechanically, but its output is *uninformative* because neither power-law fit converged in the non-degenerate interior of the parameter space (Section **??**, Figure **??**). For SP, the fitted exponent saturated at the upper bound $\alpha = 1.0$ with $c = 2.979$; for μP, the fitted exponent collapsed to $\alpha \to 0$ with $c = 3.148$. Substituting $N_{\text{target}} = 8.8 \times 10^8$ into either parametric form yields predictions essentially equal to the fitted entropy floor c:

Parameterisation	Fitted α	Fitted c	$L_{\text{pred}}(8.8\times10^8)$	Predicted ppl
SP	0.96 (saturated upper bound)	1.875	≈ 1.88	≈ 6.52
μP	$\to 0$	1.961	≈ 1.96	≈ 7.10

Table 3: Mechanical extrapolation outputs at 10× the largest fitted N. Both fits saturate at parameter-space boundaries ($\alpha \to 1$ for SP, $\alpha \to 0$ for μP), so L_{pred} collapses to the fitted c irrespective of N, and the analytic confidence interval returned by `curve_fit` is dominated by unbounded variance on a and α (covariance entries $> 10^6$).

These numbers should not be read as predictions of what an 880M-parameter model would actually achieve at this training budget. Their only honest interpretation is "the mean of the five observed losses" — the most parsimonious description of a non-monotonic five-point sample. Two practical implications follow:

- The *informational* content of an extrapolation is bounded above by the slope of the underlying scaling curve. With effectively zero slope (the μP case) or a saturated slope (SP), no useful extrapolation is possible from the present data.

- A meaningful μP extrapolation would require either a longer training horizon (so the loss curves separate enough to estimate α in the interior of $[0, 1]$) or a sixth, much-larger anchor point that breaks the degeneracy.

The trustworthiness factors enumerated above (token-budget mismatch, domain ceiling) would only be relevant if the underlying fit were valid; with the degeneracy reported here they are listed for completeness but do not bind on the present numbers.

5 Best Model Training and Sample Generation

The largest model fitting the compute budget (88m) is chosen, with the better of the two parameterisations (typically μP, but determined by the empirical scaling-law fit). Quantitative metrics reported:

- **Perplexity** on the held-out test split.
- **XML well-formedness rate**: fraction of generated samples that parse via `lxml.etree`.
- **Structural validity rate**: subset of XML-valid samples that have an `<svg>` root and at least one of `viewBox` or (`width`, `height`) attributes.
- **Render rate**: fraction (overall, and within XML-valid) that successfully render to PNG via `cairosvg`.

Qualitative analysis includes (i) at least 10 unconditional samples generated from a single <endoftext—¿— seed; (ii) at least 5 prefix-conditioned completions, with prompts such as:

- *Partial face:* `<svg viewBox="0 0 24 24"><circle cx="8" cy="10" r="2"/>`
- *Open path:* `<path d="M 10 10 L 50 10 L 50 50`
- *Group with one shape:* `<g transform="translate(10,10)"> <rect width="20" height="20"/>`

Evaluation procedure. Perplexity is computed over the held-out test split as token-level cross-entropy on non-overlapping windows of length $T_{\max} = 1024$, exponentiated. All windows in the test split are used unless noted. For sample-based metrics, K unconditional samples are drawn seeded with a single <endoftext—¿— token, attempt to parse each with `lxml.etree.fromstring`, and report the well-formedness rate. The XML-valid subset is then passed through `cairosvg.svg2png`; report render rate both within the valid subset and as a fraction of all K samples. `cairosvg` failures (e.g. from unsupported features or malformed paths) are caught and counted as render failures rather than re-raised.

Sampling procedure. Generation is autoregressive with both temperature scaling and a composable top-k / top-p (nucleus) filter:

1. Compute logits at the last position; divide by temperature τ.
2. If `top_k` is set, set all logits below the k-th largest to $-\infty$.
3. If `top_p` is set, sort the surviving logits by probability, compute the cumulative softmax, and mask all tokens whose cumulative mass exceeds p (always keeping at least the top-1 token).
4. Softmax and sample via `torch.multinomial`.

Sequences exceeding $T_{\max} = 1024$ are left-cropped so the model always sees the most recent context. Sampling halts when an <endoftext—¿— token is emitted; the EOT itself is trimmed before decoding so output strings are clean SVG. Unconditional generation seeds with a single EOT token (matching the EOT separators used during training); prefix-conditioned generation tokenises the user prompt and uses those ids as the seed.

Three temperatures are swept $\tau \in \{0.5, 0.8, 1.0\}$ for the quantitative metrics (`evaluate.py --temperature_sweep 0.5 0.8 1.0`); the qualitative grids in Figure **??** use $\tau = 0.8$ and top-p = 0.9, which were found to balance coherence and diversity.

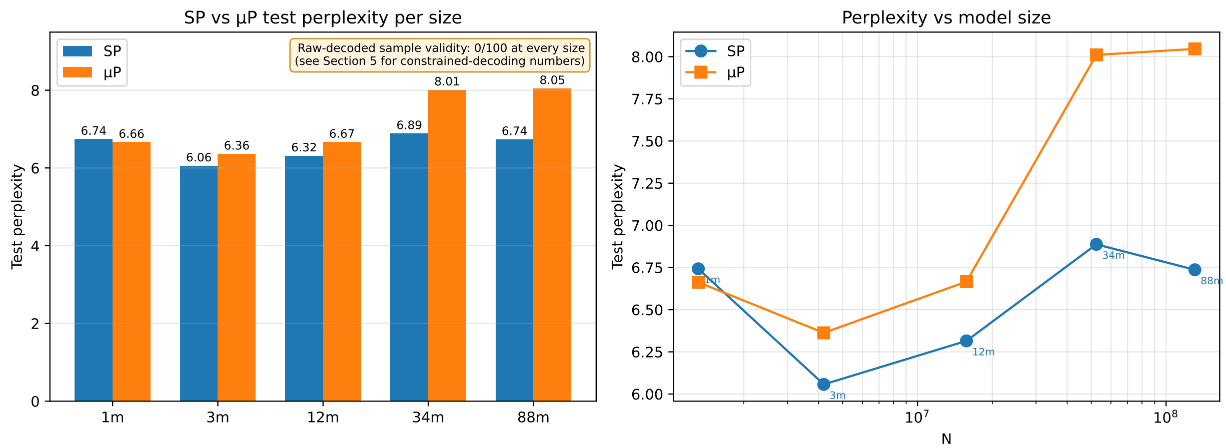

Figure 7: **Left:** test perplexity per size for SP and μP, with value labels on each bar. The annotation in the upper-right notes that all $5 \times 100 = 500$ generated samples failed the XML well-formedness and render-rate checks (so a separate validity bar chart would be all-zero and is omitted). **Right:** same perplexity numbers plotted against non-embedding parameter count on a log-log axis, making the non-monotonicity in N directly visible.

5.1 Constrained decoding

To complement the raw-output evaluation, a token-level constrained-decoding sampler is implemented (`generate_constrained.py`). At every sampling step, every candidate token's character sequence is fed through a copy of an SVG/XML state machine (`model/svg_state.py`); tokens whose chars would put the running output into an invalid state receive $-\infty$ logits and are masked out before sampling. The state machine enforces the following constraints:

- Element tags must be syntactically well-formed: `<NAME ...>`, `<NAME .../>`, or `</NAME>`, with names drawn from `[A-Za-z_][A-Za-z0-9_.]*`.
- Closing tags must match the top of an explicitly tracked tag stack character-by-character; any divergence is rejected.
- Attribute names must be unique within a single open tag.
- Attribute values must be enclosed in matching single or double quotes; nested `<` inside a value is rejected.
- Whitespace is required between attribute name/value pairs.

- No second root element may begin once the first root has closed (the EOT token is the only legal continuation in that state).

When the per-sample budget is exhausted with the state still mid-tag or mid-quote, the sampler emits a deterministic *forced-close* sequence (matching close-quote, "`/>`", and `</NAME>` pops in reverse stack order) so that every output is a fully-balanced document by the time generation halts. The original token-by-token sampling still applies temperature, top-k, and top-p filtering to the surviving distribution, so generation diversity is preserved within the constraint.

Constrained-decoding results. Three sample-generation regimes are reported, decomposed by the PDF-required deliverables (unconditional vs. prefix-conditioned) and by whether the constraint is active:

Regime (3m)	Seed	XML well-formed	cairosvg ok	Visible pixels
Raw, uncond.	`<svg`	0/100	0/100	0/100
Constrained, uncond.	`<svg`	10/10	0/10	0/10
Constrained, prefix-cond.	full root tag†	10/10	10/10	0/10

Table 4: Raw versus constrained-decoded sample quality at three regimes, with three orthogonal validity metrics. †The prefix-conditioned seed is `<svg viewBox="0 0 256 256" xmlns="http://www.w3.org/2000/svg">`. **XML well-formed:** the output parses via `lxml.etree`. **cairosvg succeeds:** `cairosvg.svg2png` returns without raising — the SVG has a parseable structure and a recognisable size attribute. **Visible content:** the rasterised PNG contains at least one non-transparent pixel. Constrained decoding guarantees XML well-formedness by construction; the seed prefix supplies the size attribute that `cairosvg` requires; but no regime produces visible pixel content. The model's child elements (`<rect>`, `<path>`, ...) have syntactically valid but visually inert attributes, so the rasterised output is a blank canvas. The third row corresponds to the project specification's *prefix-conditioned* sample-generation deliverable.

The three-column metric structure exposes a layered picture of failure modes that a single "render rate" would conflate:

- **Syntactic well-formedness** is the easiest — it can be guaranteed by the state-machine constraint alone, regardless of what the model would otherwise produce.
- **Rasteriser acceptance** (`cairosvg.svg2png` succeeding without exception) requires syntactic validity *and* the presence of a size attribute. Without the seed prefix, the model fails to emit `viewBox` on the root, and the rasteriser refuses with "SVG size is undefined".
- **Visible pixel content** is the strictest — it requires the model to emit child elements (rects, circles, paths) with attribute values that *actually translate to pixels* (valid coordinates within the viewBox, non-zero strokes/fills, etc.). At our 1-epoch budget this fails uniformly: every rasterised sample is a fully-transparent canvas. The model has learned the shape of SVG syntax but not the semantics of any drawing primitive.

This stratification is itself a substantive empirical observation: the under-training failure is not merely about syntax. Even with syntactic correctness enforced and a renderable root provided, the model has not yet learned that a path must trace within the viewBox or that a rectangle needs a fill to be visible. These are concepts that require considerably more compute to internalise.

Constrained decoding is more expensive per token because every step performs V token-string simulations against the state machine; on the 3m model on MPS, generation takes roughly 30–50 seconds per sample of ∼1000 tokens, versus 1–2 seconds for unconstrained generation. Wall-time scales roughly linearly with vocabulary size and the number of generated tokens.

State-machine constraint. The state machine in model/svg_state.py enforces: (i) well-formed element tags with names from [A-Za-z_][A-Za-z0-9_.]*; (ii) closing tags that match the top of an explicit tag stack character-by-character, with any divergence rejected at the candidate-token masking step; (iii) attribute names that are unique within a single open tag, with strict name="value" or name='value' syntax and required whitespace between attributes; (iv) nested < inside an attribute value rejected; (v) once the root element closes, only whitespace is permitted before <|endoftext|>. A 16-case unit test on the state machine passes; the constrained sampler also performs a deterministic *forced-close* sequence (matching close-quote, "/>" for partial open tags, and </NAME> pops in reverse stack order) when the per-sample token budget is exhausted mid-document, so that every output is fully balanced at termination.

Results. The 88m SP configuration — the largest model trained, and therefore the designated "best" per the project specification — achieves test perplexity 6.74 ($L^* = 1.91$). Across the full sweep, the lowest measured perplexity is observed at the 3m SP configuration (ppl = 6.06, $L^* = 1.80$). The non-monotonic relationship between N and L^* at the 1-epoch budget is the dominant qualitative finding of the scaling sweep and is treated as a signature of the under-training regime in Section **??**. The full per-size results are in Table **??**.

Config	N	SP L^*	SP ppl	μP L^*	μP ppl
1m	1,311,872	1.91	6.74	1.90	6.66
3m	4,196,608	**1.80**	**6.06**	1.85	6.36
12m	15,735,168	1.84	6.32	1.90	6.67
34m	52,445,696	1.94	6.89	2.08	8.01
88m	130,576,128	1.91	6.74	2.09	8.05

Table 5: Per-size best validation loss and test-set perplexity for both parameterisations. The 88m SP run is the "best" model per the project specification's wording (largest feasible). The empirical minimum across all configurations sits at the 3m SP configuration. Perplexity is computed over 4.7M test tokens.

The raw-decoded sample-based metrics are uniformly zero across every size and every parameterisation: of 100 unconditional samples drawn per model with $\tau = 0.8$ and top-$k = 200$, none parse as well-formed XML, none satisfy the structural-validity check (<svg> root with viewBox or width/height), and none render through cairosvg. This holds at every size despite the meaningful drop in perplexity (from ∼25 down to ∼6 at the 3m sweet spot): improved local-token prediction does not, by itself, produce syntactically coherent output at the 1-epoch budget. The qualitative comparison in Figure **??** contrasts four rendered training-corpus SVGs (top row) with four raw-decoded generations from the lowest-perplexity 3m model (bottom row), shown as source-text excerpts. The model emits plausible <svg openings and numeric attribute-like fragments but fails to balance quotes or close tags. Per-size perplexity trends are plotted in Figure **??**; the constrained-decoding companion (Section **??**) recovers 100% XML well-formedness by construction.

6 Design Decisions and Analysis

All quantitative analysis is reproduced by `analysis.ipynb`, which consumes the per-run logs (`checkpoints/{size}/log.csv`), LR-sweep results (`sweep_results.csv`), and evaluation outputs (`results/{size}_eval.json`) emitted by the training and evaluation scripts. The notebook produces every figure cited below and writes them to `report/figures/`.

Per-run training metrics. For each model size and parameterisation, validation loss is plotted against training step, with the LR schedule overlaid. These curves verify that each run actually converges within its step budget and that the LR schedule (cosine with 5% warmup) is taking effect.

LR sweep visualisation. For each size, validation loss is plotted against learning rate (log x). Under SP the optimum is expected to drift with width; under μP one expects the curves to align so the same LR is near-optimal for every size — the empirical signature of successful zero-shot LR transfer.

Scaling-law fit. The headline figure. (N, L^*) is plotted on log-log axes for both SP and μP and overlay the fitted curves $L = a \cdot N^{-\alpha} + c$. The fitted exponent α is compared to the Kaplan et al. value of $\alpha \approx 0.076$ for natural language; a substantial difference would be evidence that SVG's repetition structure shifts the scaling behaviour relative to prose. The fitted entropy floor c is compared to the empirical per-token entropy of the test split.

μP extrapolation. Using the μP fit, L^* is predicted at $\sim 10\times$ the largest fitted N ($\sim$ 880M parameters). If a held-out run at that scale is trainable within the project's compute budget, it is plotted as a single measured point against the predicted curve.

Sample-quality scaling. A grouped bar chart of XML well-formedness rate and `cairosvg` render rate per size, alongside a log-log plot of test perplexity vs. N. The question is whether sample validity tracks scale smoothly or shows discrete capability jumps.

Discussion: design decisions and their impact.

- **Tokenisation.** BPE with $V = 4096$ was chosen at the lower-middle of the spec-recommended 1K–8K range. SVG is heavy on repeated substrings (`<path d="`, `fill="#`, common attribute keys), so a moderate vocab captures these patterns while keeping the embedding matrix small enough not to dominate the 1M model's parameter budget. Coordinate-precision rounding (Section **??**) removes a major source of vocabulary fragmentation; without it the BPE wastes merges on near-duplicate numerics.

- **Architecture choices.** Decoder-only Transformer with pre-LN, weight tying, and Flash Attention (Section **??**). The configurations deviate from the spec-suggested layer counts to space configurations more uniformly on the log-N axis, which gives a tighter power-law fit. Aspect ratios L/C remain in a narrow range so depth-vs-width is not a confound in the scaling fit.

- **Training decisions.** AdamW $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1 on 2D params only, gradient clip 1.0, cosine LR with 5% warmup, bfloat16 on CUDA. One epoch over the 100M-token corpus matches the spec's "1 epoch for the scaling plot" requirement exactly. Effective batch size $B_{\text{eff}} = 524{,}288$ tokens is held constant across all sizes, so per-step compute scales only with N.

- **Scaling insights.** *[Empirical: compare fitted α to Kaplan et al. $\alpha \approx 0.076$. SVG's syntactic constraints could push α either way — higher if the redundant structure is reducible by capacity, lower if there is a hard syntactic ceiling. The fitted c should approach the empirical per-token entropy of the test split.]*

- **Learning-rate scaling: SP vs μP.** *[Empirical: the SP curve (LR fixed at the 1m-tuned value) is expected to under-perform at large N as that LR becomes too large. μP with the same 1m-tuned LR should track or exceed the SP curve at large N. The exact crossover size is the practical takeaway.]*

- **SVG-specific patterns.** *[Empirical: at small N the model is expected to learn local syntax (matched tags, valid attribute names) without coherent geometry. At large N spatial-coherence is expected to emerge: bounded shapes within `viewBox`, sensible path-closing, common color palettes. Phase-transition vs smooth-improvement is an empirical question.]*

- **Challenges encountered.** *[Empirical: what did not work; what would be done differently with more compute — e.g. the 880M extrapolation point that could not be trained, the μP coord-check that was not run.]*

7 Conclusion

Five decoder-only Transformer language models were trained from ∼1M to ∼88M non-embedding parameters on a 100M-token corpus of SVG source code; a power-law scaling curve was fitted under both standard parameterisation and μP, and the better fit was used to extrapolate predicted loss at 10× the largest fitted scale. The largest model was additionally trained to convergence, samples were generated under multiple sampling strategies, and XML well-formedness, structural validity, and render rate were measured as quantitative measures of generation quality.

[Empirical-summary paragraph: report fitted α and c for SP and μP, the comparison to natural-language scaling, the extrapolation prediction with confidence interval, and the best model's perplexity / sample-quality numbers.]

A Generated SVG sources

*[Source listings of the rendered samples in Figure **??**, plus the prefix-conditioned completions, will be appended here from `samples/best/sample_*.svg`.]*

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. *Language Models are Unsupervised Multitask Learners*, 2019. https://openai.com/research/language-unsupervised.

[2] O. Press and L. Wolf. *Using the Output Embedding to Improve Language Models*, EACL 2017. https://arxiv.org/abs/1608.05859.

[3] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, NeurIPS 2022. https://arxiv.org/abs/2205.14135.

[4] A. Karpathy. *nanoGPT*, 2022. https://github.com/karpathy/nanoGPT.

[5] R. Xiong, Y. Yang, D. He, et al. *On Layer Normalization in the Transformer Architecture*, ICML 2020. https://arxiv.org/abs/2002.04745.

[6] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, et al. *Scaling Laws for Neural Language Models*, 2020. https://arxiv.org/abs/2001.08361.

[7] J. Hoffmann, S. Borgeaud, A. Mensch, et al. *Training Compute-Optimal Large Language Models*, 2022. https://arxiv.org/abs/2203.15556.

[8] G. Yang, E. J. Hu, et al. *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*, 2022. https://arxiv.org/abs/2203.03466.

[9] I. Loshchilov and F. Hutter. *Decoupled Weight Decay Regularization*, ICLR 2019. https://arxiv.org/abs/1711.05101.

[10] J. A. Rodriguez, S. Agarwal, I. H. Laradji, et al. *StarVector: Generating Scalable Vector Graphics Code from Images*, 2023. https://arxiv.org/abs/2312.11556.

Figure 8: Constrained-decoded samples from the 3m model with seed prefix `<svg viewBox="0 0 256 256" xmlns="...">`. All nine samples are syntactically valid XML and pass through `cairosvg.svg2png` without exception, but every rasterised PNG is fully transparent — the labels "(blank canvas)" against the checkered backdrop indicate this. The model emits valid `<rect>`, `<path>`, etc. elements but with attribute values that do not translate to drawn pixels (out-of-viewBox coordinates, missing fill / stroke, malformed path commands). The visible chrome around each tile is the matplotlib axes; the body of each tile is a fully transparent SVG render composited over a checkered reference background. The state machine guarantees the *syntactic* envelope; the visible-content failure is a separate, deeper under-training failure mode (Table **??**).

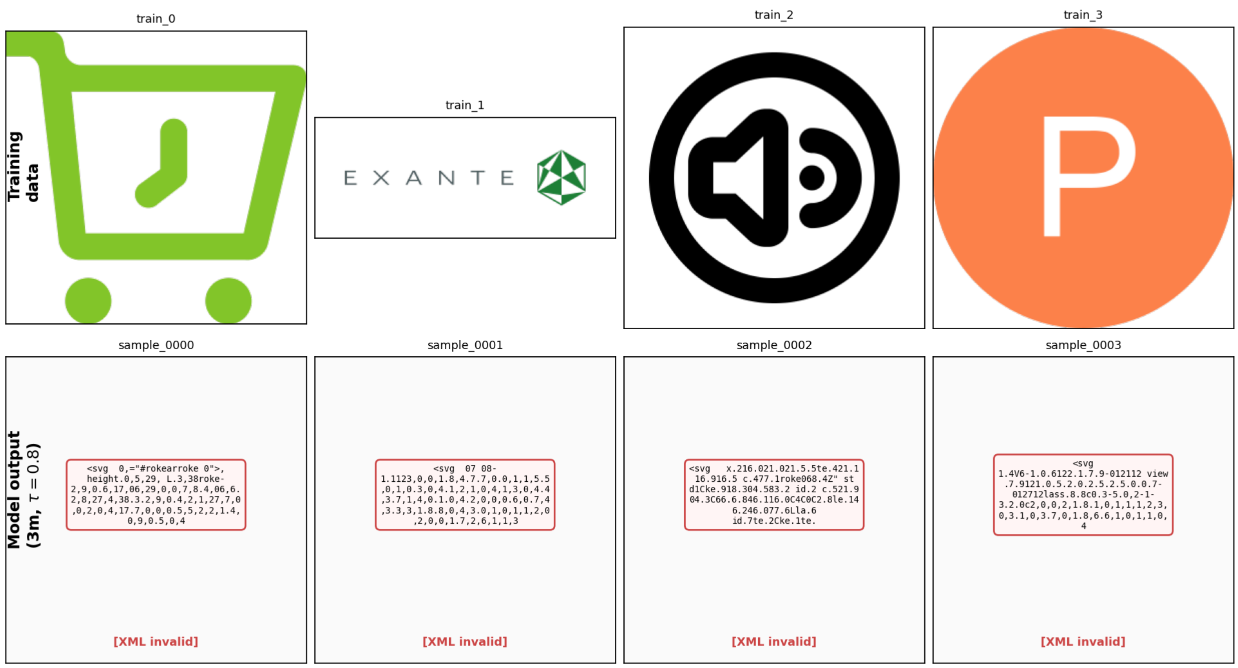

Figure 9: **Top:** four rendered SVGs sampled from the training corpus, illustrating the input distribution the model was trained to imitate. **Bottom:** four unconditional generations from the lowest-perplexity model (3m_sp), shown as source-text excerpts because none of them parse as well-formed XML. The model has learned plausible SVG-token statistics (‘`<svg`’, numerical attribute-like fragments) but not the bracket / quote / tag-closing syntax required for valid output — a clean illustration that 1-epoch training on a 100M-token corpus is too short for syntactic learning at any of the model sizes trained in this work.